Database Transactions and ACID Properties

MySQL Transaction Management

ABD

October 29, 2025

Table of Contents

What is a Transaction?

Definition

A transaction is a logical unit of work that contains one or more SQL statements. All statements in a transaction either succeed together or fail together.

Real-World Analogy

Think of withdrawing money from an ATM:

1. Check account balance
2. Deduct amount from account
3. Dispense cash
4. Update transaction log

All steps must complete, or none should!

Basic Transaction Syntax in MySQL

```sql
-- Start a transaction
START TRANSACTION;

-- Execute SQL statements
UPDATE accounts SET balance = balance - 100
WHERE account_id = 1;

UPDATE accounts SET balance = balance + 100
WHERE account_id = 2;

-- Commit the transaction
COMMIT;

-- Or rollback if something goes wrong
-- ROLLBACK;
```

Transaction Example: Bank Transfer

```sql
-- Transfer $500 from Alice to Bob
START TRANSACTION;

-- Deduct from Alice's account
UPDATE accounts
SET balance = balance - 500
WHERE customer_name = 'Alice';

-- Add to Bob's account
UPDATE accounts
SET balance = balance + 500
WHERE customer_name = 'Bob';

-- Make it permanent
COMMIT;
```

What if power fails between the two UPDATEs?

Without transactions: Money disappears!

With transactions: Everything rolls back automatically.

The ACID Properties

ACID Guarantees

ACID is an acronym that describes four critical properties that ensure database transactions are processed reliably:

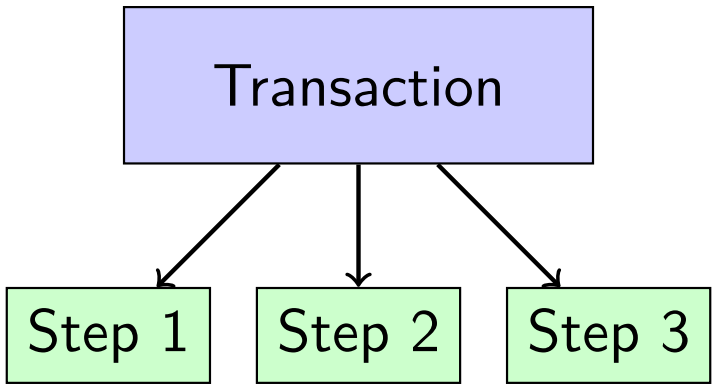
- **A**tomicity
- **C**onsistency
- **I**solation
- **D**urability

These properties work together to maintain data integrity even in the face of errors, power failures, and concurrent access.

Atomicity: All or Nothing

Definition

Atomicity ensures that a transaction is treated as a single, indivisible unit. Either all operations within the transaction succeed, or none of them do.



All succeed or all fail

Atomicity Example 1: E-commerce Order

```
START TRANSACTION;

-- Insert order
INSERT INTO orders (customer_id, order_date, total)
VALUES (101, NOW(), 150.00);

-- Get the order ID
SET @order_id = LAST_INSERT_ID();

-- Insert order items
INSERT INTO order_items (order_id, product_id,
    quantity, price)
VALUES (@order_id, 5001, 2, 50.00);

INSERT INTO order_items (order_id, product_id,
    quantity, price)
VALUES (@order_id, 5002, 1, 50.00);
```

Atomicity Example1: E-commerce Order

```
1  -- Update inventory
2  UPDATE products SET stock = stock - 2
3  WHERE product_id = 5001;
4
5  UPDATE products SET stock = stock - 1
6  WHERE product_id = 5002;
7
8  COMMIT;
```

Atomicity Example 2: Rollback on Error

```sql
START TRANSACTION;

UPDATE accounts SET balance = balance - 1000
WHERE account_id = 1;

-- Oops! This fails due to insufficient funds
UPDATE accounts SET balance = balance + 1000
WHERE account_id = 2 AND balance >= 0;

-- Check for errors in application code
-- If error detected:
ROLLBACK; -- Undoes all changes!

-- Account 1 is NOT debited because
-- the entire transaction is rolled back
```

Result

Without rollback, account 1 would lose money but account 2 wouldn't receive it!

Atomicity Example 3: SAVEPOINT

```
1  START TRANSACTION;
2
3  UPDATE accounts SET balance = balance - 100
4  WHERE account_id = 1;
5
6  SAVEPOINT sp1;
7
8  UPDATE accounts SET balance = balance - 50
9  WHERE account_id = 1;
10
11 SAVEPOINT sp2;
```

Atomicity Example 3: SAVEPOINT

```
1 UPDATE accounts SET balance = balance - 25
2 WHERE account_id = 1;
3
4 -- Oops, last update was wrong
5 ROLLBACK TO SAVEPOINT sp2;
6 -- Only the -25 is undone
7
8 COMMIT; -- Commits the -100 and -50
```

Consistency: Valid State Transitions

Definition

Consistency ensures that a transaction brings the database from one valid state to another, maintaining all defined rules, constraints, and triggers.

Database Rules Include:

- Primary key constraints
- Foreign key constraints
- CHECK constraints
- Data type constraints
- Trigger validations
- Application-level business rules

Consistency Example 1: Constraint Enforcement

```sql
-- Table with CHECK constraint
CREATE TABLE accounts (
    account_id INT PRIMARY KEY,
    balance DECIMAL(10,2),
    CONSTRAINT balance_positive CHECK (balance >= 0)
);

START TRANSACTION;

-- This will FAIL and rollback
UPDATE accounts SET balance = balance - 500
WHERE account_id = 1 AND balance = 300;
-- ERROR: Check constraint violated!

ROLLBACK; -- Automatic in MySQL on constraint violation
```

Consistency Example 1: Constraint Enforcement

Consistency Maintained

The database won't allow negative balances, maintaining data validity.

Consistency Example 2: Foreign Key Constraints

```
1  CREATE TABLE customers (
2      customer_id INT PRIMARY KEY,
3      name VARCHAR(100)
4  );
5
6  CREATE TABLE orders (
7      order_id INT PRIMARY KEY,
8      customer_id INT,
9      FOREIGN KEY (customer_id) REFERENCES customers(
           customer_id)
10 );
```

Consistency Example 2: Foreign Key Constraints

```
START TRANSACTION;

-- This will FAIL
INSERT INTO orders (order_id, customer_id)
VALUES (1001, 999);
-- ERROR: Foreign key constraint fails!
-- Customer 999 doesn't exist

ROLLBACK;
```

Consistency Example 3: Business Logic

```sql
-- Enforce: Total debits must equal total credits
START TRANSACTION;

-- Debit account
INSERT INTO journal_entries
(account_id, type, amount)
VALUES (1, 'DEBIT', 1000);

-- Credit account
INSERT INTO journal_entries
(account_id, type, amount)
VALUES (2, 'CREDIT', 1000);
```

Consistency Example 3: Business Logic

```
1  -- Verify balance (in application or trigger)
2  SELECT
3      SUM(CASE WHEN type='DEBIT' THEN amount ELSE 0
           END) as debits,
4      SUM(CASE WHEN type='CREDIT' THEN amount ELSE 0
           END) as credits
5  FROM journal_entries;
6
7  -- If debits != credits, ROLLBACK
8  COMMIT;
```

Consistency Example 4: Using Triggers

```
1  DELIMITER //
2  CREATE TRIGGER check_stock_before_order
3  BEFORE INSERT ON order_items
4  FOR EACH ROW
5  BEGIN
6      DECLARE current_stock INT;
7
8      SELECT stock INTO current_stock
9      FROM products
10     WHERE product_id = NEW.product_id;
11
12     IF current_stock < NEW.quantity THEN
13         SIGNAL SQLSTATE '45000'
14         SET MESSAGE_TEXT = 'Insufficient stock';
15     END IF;
16 END//
17 DELIMITER ;
```

Consistency Example 4: Using Triggers

```
-- Transaction will fail if stock is insufficient
START TRANSACTION;
INSERT INTO order_items (product_id, quantity)
VALUES (101, 100); -- Fails if stock < 100
COMMIT;
```

Isolation: Concurrent Transaction Control

Definition

Isolation ensures that concurrent execution of transactions leaves the database in the same state as if the transactions were executed sequentially.

The Problem

Multiple users accessing the database simultaneously can cause:

- **Dirty Reads**: Reading uncommitted data
- **Non-repeatable Reads**: Data changes between reads
- **Phantom Reads**: New rows appear between reads
- **Lost Updates**: Concurrent updates overwrite each other

Isolation Levels in MySQL

Level	**Dirty**	**Non-rep.**	**Phantom**
READ UNCOMMITTED	Yes	Yes	Yes
READ COMMITTED	No	Yes	Yes
REPEATABLE READ	No	No	No*
SERIALIZABLE	No	No	No

- MySQL default: **REPEATABLE READ**
- *InnoDB prevents phantoms in REPEATABLE READ
- Higher isolation = More locks = Less concurrency

Isolation Example 1: Dirty Read Problem

Transaction 1:

```sql
START TRANSACTION;

UPDATE accounts
SET balance = 5000
WHERE id = 1;

-- Not committed yet!
-- Working on other things...

-- Oops, error!
ROLLBACK;
```

Transaction 2:

```sql
SET TRANSACTION
ISOLATION LEVEL
READ UNCOMMITTED;

START TRANSACTION;

SELECT balance
FROM accounts
WHERE id = 1;
-- Returns 5000!
-- But this is DIRTY data
-- (will be rolled back)

COMMIT;
```

Problem

Transaction 2 reads data that never actually existed in committed state!

Isolation Example 2: Non-Repeatable Read

Transaction 1:

```sql
SET TRANSACTION
ISOLATION LEVEL
READ COMMITTED;

START TRANSACTION;

SELECT balance
FROM accounts
WHERE id = 1;
-- Returns 1000

-- ... some time passes

SELECT balance
FROM accounts
WHERE id = 1;
-- Returns 1500!
-- Different value!

COMMIT;
```

Transaction 2:

```sql


START TRANSACTION;

-- Runs between T1's
-- two SELECTs

UPDATE accounts
SET balance = 1500
WHERE id = 1;

COMMIT;
```

Problem

Same SELECT query returns different results within one transaction!

Isolation Example 3: Phantom Read

Transaction 1:

```sql
SET TRANSACTION
ISOLATION LEVEL
READ COMMITTED;

START TRANSACTION;

SELECT COUNT(*)
FROM orders
WHERE status='PENDING';
-- Returns 5

-- ... some processing

SELECT COUNT(*)
FROM orders
WHERE status='PENDING';
-- Returns 7!
-- New rows appeared!

COMMIT;
```

Transaction 2:

```sql


START TRANSACTION;

INSERT INTO orders
(status)
VALUES ('PENDING');

INSERT INTO orders
(status)
VALUES ('PENDING');

COMMIT;
```

Problem

New rows appear ("phantoms") that weren't there before in the same transaction!

Isolation Example 4: Lost Update Problem

Transaction 1:

```sql
START TRANSACTION;

SELECT balance
FROM accounts
WHERE id = 1;
-- Returns 1000

-- Calculate new balance
-- new_balance = 1000 + 500

UPDATE accounts
SET balance = 1500
WHERE id = 1;

COMMIT;
```

Transaction 2:

```sql
START TRANSACTION;

SELECT balance
FROM accounts
WHERE id = 1;
-- Returns 1000

-- Calculate new balance
-- new_balance = 1000 + 300

UPDATE accounts
SET balance = 1300
WHERE id = 1;

COMMIT;
```

Problem

Both read 1000, but final balance is 1300 instead of 1800. One update is lost!

Isolation Example 5: Fixing with Locking

```sql
START TRANSACTION;

-- Lock the row for update
SELECT balance FROM accounts
WHERE id = 1
FOR UPDATE;
-- Returns 1000, and LOCKS the row

-- Other transactions must wait!

UPDATE accounts
SET balance = balance + 500
WHERE id = 1;

COMMIT; -- Releases the lock
```

Better Approach

Use `SET balance = balance + 500` instead of reading then updating!

Isolation Example 6: Setting Isolation Level

```
1 -- Set for current session
2 SET SESSION TRANSACTION ISOLATION LEVEL
3 REPEATABLE READ;
4
5 -- Set for next transaction only
6 SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
7
8 START TRANSACTION;
9 -- This transaction uses SERIALIZABLE
```

Isolation Example 5: Setting Isolation Level

```
1  SELECT balance FROM accounts WHERE id = 1;
2  -- In SERIALIZABLE: Locks for reads too!
3
4  COMMIT;
5
6  -- Check current isolation level
7  SELECT @@transaction_ISOLATION;
8  -- Or
9  SHOW VARIABLES LIKE 'transaction_isolation';
```

Durability: Permanent Changes

Definition

Durability ensures that once a transaction is committed, its changes are permanent and survive system failures, crashes, or power outages.

Implementation Mechanisms

- **Write-Ahead Logging (WAL)**: Changes written to log before data files
- **Transaction Log**: Sequential log of all changes
- **Checkpoints**: Periodic flushing to disk
- **Recovery Protocols**: Replay logs after crash

Durability Example 1: Understanding the Guarantee

```
1  START TRANSACTION;
2
3  INSERT INTO critical_data (value, timestamp)
4  VALUES ('Important', NOW());
5
6  COMMIT;
7  -- At this point, MySQL guarantees the data is safe
       !
8
9  -- Even if:
10 -- - Server crashes immediately after
11 -- - Power goes out
12 -- - Disk fails (with proper RAID/backup)
13 -- The INSERT will be there when system restarts
```

Durability Example 1: Understanding the Guarantee

What MySQL Does

1. Writes to transaction log (redo log)
2. Writes to data files
3. Syncs to physical disk
4. Only then returns success to application

Durability Example 2: InnoDB Settings

```
1  -- Check durability settings
2  SHOW VARIABLES LIKE 'innodb_flush_log_at_trx_commit
       ';
3
4  -- Values:
5  -- 0 = Flush every second (FAST, less durable)
6  -- 1 = Flush at each commit (SLOW, most durable)
7  -- 2 = Flush to OS cache (MEDIUM)
8
9  -- Set for maximum durability
10 SET GLOBAL innodb_flush_log_at_trx_commit = 1;
11
12 -- Check sync settings
13 SHOW VARIABLES LIKE 'sync_binlog';
14 -- 1 = Sync binary log at each commit
```

Trade-off

More durability = More disk I/O = Slower performance

Durability Example 3: Testing Recovery

```
1 -- In Terminal 1: Start a transaction
2 START TRANSACTION;
3
4 INSERT INTO test_table (data) VALUES ('Before crash
      ');
5 INSERT INTO test_table (data) VALUES ('Before crash
       2');
6
7 COMMIT;
```

Durability Example 3: Tesing Recovery

```
SELECT * FROM test_table;
-- Shows 2 rows

-- Simulate crash (don't do this in production!)
-- sudo kill -9 $(pidof mysqld)

-- After MySQL restarts:
SELECT * FROM test_table;
-- Still shows 2 rows! Data survived!
```

Durability Example 4: Binary Logging

```
-- Enable binary logging (in my.cnf)
-- log-bin=mysql-bin
-- server-id=1

-- View binary logs
SHOW BINARY LOGS;

-- See what's in the log
SHOW BINLOG EVENTS IN 'mysql-bin.000001';

-- Binary logs used for:
-- 1. Point-in-time recovery
-- 2. Replication
-- 3. Auditing

-- These provide additional durability layer
```

Complete Example: Airline Reservation System

```sql
START TRANSACTION;

-- Check seat availability
SELECT seat_status FROM seats
WHERE flight_id = 'AA100' AND seat_number = '12A'
FOR UPDATE;

-- If available (checked in application):
-- Reserve the seat
UPDATE seats
SET seat_status = 'RESERVED', customer_id = 501
WHERE flight_id = 'AA100' AND seat_number = '12A';

-- Create reservation
INSERT INTO reservations (customer_id, flight_id, seat_number, booking_date)
VALUES (501, 'AA100', '12A', NOW());

-- Process payment
INSERT INTO payments (customer_id, amount, payment_date)
VALUES (501, 299.99, NOW());

-- Update customer's total spent
UPDATE customers SET total_spent = total_spent + 299.99
WHERE customer_id = 501;

COMMIT;
```

Complete Example: Inventory Management

```sql
START TRANSACTION;

-- Check current inventory
SELECT quantity INTO @current_qty FROM inventory
WHERE product_id = 2001 AND warehouse_id = 5
FOR UPDATE;

-- Verify sufficient quantity
-- (Done in application: if @current_qty < 50 then ROLLBACK)

-- Deduct from source warehouse
UPDATE inventory
SET quantity = quantity - 50, last_updated = NOW()
WHERE product_id = 2001 AND warehouse_id = 5;

-- Add to destination warehouse
INSERT INTO inventory (product_id, warehouse_id, quantity, last_updated)
VALUES (2001, 8, 50, NOW())
ON DUPLICATE KEY UPDATE
    quantity = quantity + 50,
    last_updated = NOW();

-- Log the transfer
INSERT INTO inventory_transfers
(product_id, from_warehouse, to_warehouse, quantity, transfer_date)
VALUES (2001, 5, 8, 50, NOW());

COMMIT;
```

Complete Example: Multi-Currency Exchange

```sql
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
START TRANSACTION;

-- Get current exchange rate
SELECT rate INTO @exchange_rate FROM exchange_rates
WHERE from_currency = 'USD' AND to_currency = 'EUR'
AND effective_date <= NOW()
ORDER BY effective_date DESC LIMIT 1
FOR SHARE;

-- Deduct USD
UPDATE accounts
SET balance = balance - 1000
WHERE account_id = 100 AND currency = 'USD';

-- Add EUR
UPDATE accounts
SET balance = balance + (1000 * @exchange_rate)
WHERE account_id = 100 AND currency = 'EUR';

-- Record transaction
INSERT INTO currency_exchanges
(account_id, from_curr, to_curr, amount, rate, exchange_date)
VALUES (100, 'USD', 'EUR', 1000, @exchange_rate, NOW());

COMMIT;
```

Error Handling Pattern

```
1  DELIMITER //
2  CREATE PROCEDURE safe_transfer(
3      IN from_acc INT, IN to_acc INT, IN amount
           DECIMAL(10,2)
4  )
5  BEGIN
6      DECLARE EXIT HANDLER FOR SQLEXCEPTION
7      BEGIN
8          ROLLBACK;
9          SELECT 'Transaction rolled back due to
               error' AS message;
10     END;
11
12     START TRANSACTION;
13
14     UPDATE accounts SET balance = balance - amount
15     WHERE account_id = from_acc;
```

Error Handling Pattern

```
    UPDATE accounts SET balance = balance + amount
    WHERE account_id = to_acc;

    COMMIT;
    SELECT 'Transaction successful' AS message;
END//
DELIMITER ;

-- Call it
CALL safe_transfer(1, 2, 100.00);
```

Transaction Best Practices

1. **Keep transactions short**
   - Reduces lock contention
   - Improves concurrency
2. **Use appropriate isolation levels**
   - Don't use SERIALIZABLE unless necessary
   - REPEATABLE READ is usually sufficient
3. **Handle errors properly**
   - Always have ROLLBACK on error
   - Use stored procedures with error handlers
4. **Use FOR UPDATE carefully**
   - Only lock what you need to modify
   - Consider FOR SHARE for read locks
5. **Avoid user interaction in transactions**
   - Never wait for user input mid-transaction
   - Prepare data first, then transact

Common Pitfalls to Avoid

Mistakes to Avoid

- **Long-running transactions**: Hold locks too long
- **Implicit commits**: DDL statements auto-commit!
- **Not using transactions**: For multi-statement operations
- **Wrong isolation level**: Too high (slow) or too low (incorrect)
- **Forgetting COMMIT**: Locks held until connection closes
- **Nested transactions**: MySQL doesn't support true nesting
- **Autocommit=1**: Each statement commits automatically

Checking Transaction Status

```
-- View current transactions
SELECT * FROM information_schema.innodb_trx;

-- View current locks
SELECT * FROM performance_schema.data_locks;

-- View lock waits
SELECT * FROM performance_schema.data_lock_waits;

-- Kill a problematic transaction
KILL <thread_id>;

-- Check autocommit status
SELECT @@autocommit;

-- Disable autocommit for session
SET autocommit = 0;
```

ACID Properties Summary

Atomicity

All or nothing execution - transactions cannot be partially completed

Consistency

Database moves from one valid state to another - all constraints satisfied

Isolation

Concurrent transactions don't interfere with each other

Durability

Committed changes survive system failures and persist permanently

Key Takeaways

- Transactions are essential for data integrity
- ACID properties work together to ensure reliability
- MySQL InnoDB provides full ACID compliance
- Choose appropriate isolation levels for your use case
- Always handle errors and rollback when needed
- Test transaction behavior under concurrent load
- Monitor long-running transactions and locks

Questions?